

AI - POWERED URL SECURITY AND PHISHING DETECTION SYSTEM

A PROJECT REPORT

Submitted by

ABISHEK G P (910622104003)

AKASH A (910622104008)

ESWAR M S (910622104031)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



K.L.N. COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2026

K.L.N. COLLEGE OF ENGINEERING

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report “**AI-POWERED URL SECURITY AND PHISHING DETECTION SYSTEM**” is the bonafide work of “**ABISHEK G P (910622104003)**”, “**AKASH A (910622104008)**”, “**ESWAR M S (910622104031)**” who carried out the project under my supervision.

SIGNATURE

Dr. A. ASKARUNISA

HEAD OF THE DEPARTMENT

Computer Science and Engineering

SIGNATURE

Mrs. A. AKILESHWARI

SUPERVISOR

ASSISTANT PROFESSOR

Computer Science and Engineering

Submitted for the project viva-voice conducted on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be but incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We extend our gratitude to the Founder, **Late. Thiru. K.L.N. KRISHNAN, K.L.N College of Engineering and Management Members** for making us march towards the glory of success. We express our sincere thanks to our respected Principal **Dr. A.V. RAM PRASAD, M.E., Ph.D.**, for all the facilities offered.

We would like to express our deep gratitude and heartfelt thanks to **Dr. A. ASKARUNISA, M.E., Ph.D.**, Head of the Department of Computer Science and Engineering, **K.L.N. College of Engineering**, who motivated and encouraged us to do this out rival project for this academic year.

Our profound, delightful and sincere thanks to our Project guide **Mrs. A. AKILESHWARI, M.E.(CSE).**, Assistant Professor and our respected Project Coordinator **Mrs. V. KARPAGAM, M.E.(CSE), Ph.D.**, Associate Professor of the Department of Computer Science and Engineering, **K.L.N. College of Engineering** whose support was inevitable during the entire period of our work.

We thank our teaching staffs for sharing their knowledge and view to enhance our project. We also thank our non-teaching staff for extending their technical support to us. We thank our parents for giving us such a wonderful life our my friends for their friendly encouragement throughout the project. Finally, we thank the Almighty for giving the full health to finish the Project successfully.

ABSTRACT

The objective of this project is to detect and classify URLs as safe, suspicious, or malicious using AI and cybersecurity techniques. Users submit URLs via a Flask-based web interface, where the system normalizes and analyzes them for structural and lexical features such as domain length, HTTPS usage, IP addresses, redirection patterns, and suspicious keywords. A machine learning model using XGBoost is trained on a large dataset of phishing and legitimate URLs, achieving an accuracy of 97.8%. External threat intelligence from VirusTotal and Google Safe Browsing APIs is integrated for real-time validation.

Additionally, the system performs SSL certificate verification, WHOIS lookups, IP geolocation, and homograph detection to identify sophisticated phishing tactics. Each URL is assigned a threat score and classification, stored in a SQLite database, with administrative functionality to manually label scans. Users and admins can view scan history and download detailed CSV or PDF reports for analysis.

Combining XGBoost-based ML with cybersecurity checks, the system provides accurate phishing detection and is suitable for online banking, financial services, education, and general web browsing, helping protect users from phishing attacks, identity theft, and financial fraud.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
	LIST OF SYMBOLS	x
1.	INTRODUCTION	1
	1.1 XGBOOST ALGORITHM	1
	1.2 AI-POWERED URL SECURITY AND PHISHING DETECTION SYSTEM	3
	1.3 FEATURES OF AI-POWERED URL SECURITY AND PHISHING DETECTION SYSTEM	5
	1.3.1 Real-Time URL Scanning	5
	1.3.2 URL Feature Extraction & Analysis	5
	1.3.3 Threat Intelligence Integration	6
	1.3.4 IP Address Analysis	6
	1.3.5 WHOIS Domain Information Lookup	6
	1.3.6 SSL Certificate Validation	6
	1.3.7 Redirect & Domain Change Analysis	6
	1.3.8 Subdomain & Homograph Attack Detection	7
	1.3.9 Threat Scoring Mechanism	7
	1.3.10 Database Storage and Admin Panel	7
	1.3.11 Interactive Web Interface	7
2.	LITERATURE REVIEW	8
3.	SYSTEM ANALYSIS	13
	3.1 EXISTING SYSTEM	13
	3.1.1 Disadvantages	13
	3.1.2 Scalability Challenges	14
	3.2 PROPOSED SYSTEM	14
	3.2.1 Advantages	16
	3.2.2 Overcoming Existing Limitations	16

4.	SYSTEM REQUIREMENTS	17
	4.1 HARDWARE REQUIREMENTS	17
	4.2 SOFTWARE REQUIREMENTS	17
	4.3 HARDWARE DESCRIPTION	18
	4.3.1 Processor	18
	4.3.2 Memory	18
	4.3.3 Storage	19
	4.4 SOFTWARE DESCRIPTION	19
	4.4.1 Development Environment	19
	4.4.2 Operating System	19
	4.4.3 Programming Language	19
	4.4.4 Libraries	20
	4.4.5 Web Technologies	22
	4.4.6 APIs	22
	4.4.7 Web Browser	22
5.	SYSTEM DESIGN	23
	5.1 SYSTEM ARCHITECTURE	23
	5.2 DATA FLOW DIAGRAM	24
	5.3 USE CASE DIAGRAM	26
6.	SYSTEM IMPLEMENTATION	27
	6.1 LIST OF MODULES	27
	6.2 MODULE DESCRIPTION	27
	6.2.1 User Interface Module	27
	6.2.2 URL Processing & Normalization Module	28
	6.2.3 Feature Extraction & Heuristic Module	28
	6.2.4 External Threat Intelligence Module	28
	6.2.5 Network & Security Analysis Module	29
	6.2.6 Domain & IP Intelligence Module	29
	6.2.7 Data Storage Module	30
	6.2.8 Admin Module	30
7.	RESULT AND DISCUSSION	31
8.	SAMPLE CODE	35

9.	SYSTEM TESTING	47
	9.1 Unit Testing	47
	9.2 Functional Testing	48
	9.3 Integration Testing	48
	9.4 Performance Testing	48
	9.5 Security Testing	49
	9.6 User Interface Testing	49
	9.7 Validation Testing	49
10.	CONCLUSION	50
11.	FUTURE ENHANCEMENT	51
12.	REFERENCES	52

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1	Intel i5 Processor	18
4.2	8GB DDR4 RAM	18
4.3	512GB SSD	19
5.1	Architecture Diagram	23
5.2	Flow Diagram	24
5.3	Use Case Diagram	26
7.1	Entering the URL	31
7.2	Safe Website Result	32
7.3	Suspicious Website Result	32
7.4	Phishing Website Result	33
7.5	WHOIS Information Result	33
7.6	SSL Certification Result	34

LIST OF ABBREVIATIONS

ABBREVIATIONS	EXPANSION
XGBoost	Extreme Gradient Boosting
AI	Artificial Intelligence
ML	Machine Learning
URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol
SSL	Secure Sockets Layer
TLS	Transport Layer Security
IP	Internet Protocol
DNS	Domain Name System
JSON	JavaScript Object Notation
ISP	Internet Service Provider
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
VT	VirusTotal
GSB	Google Safe Browsing

LIST OF SYMBOLS

SYMBOL

SYMBOL NAME



Process / Operation Block



Input / Output Block



Flow Lines



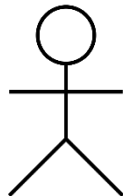
Parallel Process



Database



Use Case



Actor

CHAPTER 1

INTRODUCTION

1.1 XGBOOST ALGORITHM

The rapid growth of the internet has led to a significant increase in cyber threats, particularly phishing attacks, where malicious websites impersonate legitimate ones to steal sensitive user information such as passwords, banking details, and personal data. Detecting such fraudulent URLs in real time has become a critical challenge in cybersecurity. To address this issue, the proposed system utilizes an advanced machine learning algorithm known as XGBoost (Extreme Gradient Boosting) for accurate and efficient phishing detection. This project utilizes a single CNN model through the DeepFace library for real-time facial emotion recognition. The system can process images captured from a webcam or uploaded by the user. Each image or frame is first preprocessed by converting it to grayscale, detecting faces using a Haar Cascade classifier, and normalizing the region of interest. The CNN then analyzes the face to determine the dominant emotion, such as happy, sad, neutral, or surprised, along with the gender of the individual. By employing a single CNN, the system achieves efficient and accurate emotion detection while remaining lightweight enough for real-time processing.

XGBoost is a powerful ensemble learning algorithm based on the gradient boosting framework. It works by combining multiple weak learners, typically decision trees, to form a strong predictive model. Unlike traditional machine learning algorithms, XGBoost builds models sequentially, where each new model attempts to correct the errors made by the previous ones. This iterative learning approach significantly improves prediction accuracy. One of the key advantages of XGBoost is its ability to handle large-scale data efficiently while maintaining high speed and performance, making it highly suitable for real-time applications such as URL security analysis.

In this project, XGBoost is used to classify URLs as either safe or malicious based on various extracted features. When a user inputs a URL into the system, it undergoes a feature extraction process where important characteristics are analysed. These features include URL length, presence of special characters (such as '@', '-', or multiple subdomains), use of HTTPS protocol, domain age, redirection behaviour, and other lexical and host-based attributes. These features serve as input to the XGBoost model, which has been trained on a large dataset containing both legitimate and phishing URLs.

The strength of XGBoost lies in its ability to capture complex patterns and relationships between features that may not be easily detected by simpler algorithms. It uses gradient descent optimization to minimize prediction errors and includes regularization techniques to prevent overfitting, ensuring that the model performs well on unseen data. Additionally, XGBoost supports parallel processing and tree pruning, which enhances computational efficiency and reduces training time. These characteristics make it an ideal choice for cybersecurity applications where both accuracy and speed are crucial.

Another important aspect of XGBoost is its robustness in handling missing data and noisy inputs, which are common in real-world URL datasets. The algorithm automatically learns the best way to handle missing values, improving model reliability. Furthermore, its feature importance capability helps in identifying which URL characteristics contribute most to phishing detection, providing insights into attack patterns and improving system transparency.

In this AI-powered URL Security and Phishing Detection System, the trained XGBoost model is integrated with a web-based interface developed using Flask. When a user submits a URL, the system processes it in real time, extracts relevant features, and passes them to the XGBoost classifier. The model then predicts whether the URL is safe or potentially harmful and assigns a risk level accordingly. The result is displayed to the user in a clear and interactive format, helping them make informed decisions before accessing the website.

Overall, the use of XGBoost in this project enhances the system's capability to detect phishing attacks with high accuracy, efficiency, and scalability. By

leveraging advanced machine learning techniques, the system provides a proactive defense mechanism against malicious URLs, contributing to safer web browsing and improved cybersecurity awareness.

1.2. AI-POWERED URL SECURITY AND PHISHING DETECTION SYSTEM

The AI-Powered URL Security and Phishing Detection System with Threat Intelligence Integration is an advanced cybersecurity solution designed to detect malicious and phishing websites in real time by combining machine learning techniques with external security intelligence services. With the rapid growth of digital platforms, users increasingly rely on the internet for sensitive activities such as online banking, communication, and e-commerce. However, this has also led to a rise in sophisticated phishing attacks, where attackers create deceptive websites that closely resemble legitimate ones to steal confidential information. To address this challenge, the proposed system provides a multi-layered approach that integrates intelligent URL analysis with real-time threat intelligence, ensuring enhanced accuracy and reliability in detecting harmful websites.

The system operates through a structured pipeline that begins with user input, where a URL is provided through an interactive web interface. Once the URL is received, it is normalized and processed to extract various lexical and structural features such as URL length, presence of suspicious keywords, number of subdomains, and usage of secure protocols. These features are analyzed using a scoring-based mechanism inspired by machine learning principles and can be further enhanced using models like XGBoost, which are capable of identifying complex patterns associated with phishing behavior. This internal analysis forms the first layer of detection, enabling the system to quickly evaluate whether a URL exhibits characteristics commonly found in malicious websites.

In addition to internal analysis, the system integrates powerful external threat intelligence services such as VirusTotal and Google Safe Browsing to enhance detection accuracy. VirusTotal aggregates data from multiple antivirus

engines and security vendors to determine whether a URL is associated with malware or phishing activities. Similarly, Google Safe Browsing provides up-to-date information about unsafe websites involved in social engineering or malware distribution. By querying these APIs in real time, the system gains access to a global database of known threats, allowing it to identify malicious URLs that may not be detected through local analysis alone. This integration ensures that the system remains current with evolving cyber threats and significantly reduces false negatives.

Furthermore, the system incorporates additional security checks such as IP address analysis, WHOIS domain information retrieval, SSL certificate validation, and redirection behavior analysis. These features provide deeper insights into the legitimacy of a website. For instance, WHOIS data helps determine domain age and ownership, while SSL validation ensures secure communication channels. Redirect analysis identifies suspicious behavior where users are silently redirected to different domains, a common tactic in phishing attacks. The combination of these techniques creates a comprehensive evaluation framework that strengthens the system's ability to detect even highly sophisticated threats.

The backend of the system is implemented using Python with the Flask framework, enabling seamless integration between the user interface and the detection engine. The system processes requests efficiently and provides real-time feedback, displaying results such as threat classification (safe, suspicious, or phishing), threat score, and detailed analysis reports. Additionally, all scanned URLs and their results are stored in a database, allowing for tracking, monitoring, and administrative analysis. This feature adds an extra layer of functionality, making the system suitable for both individual users and organizational use.

In conclusion, the proposed system represents a hybrid approach to phishing detection by combining AI-based URL analysis with real-time threat intelligence integration. This dual-layered strategy enhances detection accuracy, improves system reliability, and ensures adaptability to emerging cyber threats.

By leveraging both machine learning concepts and globally recognized security APIs, the system provides a robust and scalable solution for protecting users against malicious websites and ensuring safer online experiences.

1.3. FEATURES OF THE AI-POWERED URL SECURITY AND PHISHING DETECTION SYSTEM

The AI-Powered URL Security and Phishing Detection System incorporates a comprehensive set of features that enable accurate, real-time identification of malicious and phishing websites. These features are designed to combine machine learning-based analysis with network-level inspection and real-time threat intelligence, ensuring a robust and multi-layered security approach. By integrating various detection techniques such as URL feature analysis, IP inspection, domain intelligence, SSL validation, and external API-based threat verification, the system provides reliable and detailed insights into the safety of a given URL.

1.3.1. Real-Time URL Scanning

The system provides real-time URL scanning, allowing users to instantly analyze any website through a web-based interface. When a user inputs a URL, the system processes it immediately and performs multiple security checks without noticeable delay. This feature ensures quick detection of potentially harmful websites and helps users make safe browsing decisions before accessing unknown links.

1.3.2. URL Feature Extraction & Analysis

The system analyzes various structural and lexical properties of the URL, such as its length, number of dots, presence of special characters, suspicious keywords (like login, verify, banking), and HTTPS usage. These features are evaluated using a scoring mechanism inspired by machine learning techniques and can be enhanced using models like XGBoost to detect hidden phishing patterns effectively.

1.3.3. Threat Intelligence Integration (APIs)

A major feature of the system is the integration of external threat intelligence services such as VirusTotal and Google Safe Browsing. These APIs provide real-time information about known malicious URLs and phishing domains. By combining internal analysis with global threat databases, the system significantly improves detection accuracy and protects against newly emerging cyber threats.

1.3.4. IP Address Analysis

The system extracts the IP address of the given URL and analyzes additional information such as geographical location, city, and Internet Service Provider (ISP). It also checks whether the URL directly uses an IP address instead of a domain name, which is a common indicator of phishing. This feature enhances detection by adding a network-level security check.

1.3.5. WHOIS Domain Information Lookup

The system includes WHOIS functionality to retrieve domain registration details such as registrar name, creation date, and expiration date. Newly registered or suspicious domains are often linked to phishing attacks, and this feature helps identify such domains by analyzing their history and ownership.

1.3.6. SSL Certificate Validation

The system performs SSL certificate validation to ensure that the website uses a secure connection. It checks certificate details such as issuer, validity period, and expiration date. Websites with expired or missing SSL certificates are flagged as unsafe, helping users avoid insecure connections.

1.3.7. Redirect & Domain Change Analysis

The system analyzes URL redirection behavior to detect suspicious activity. It tracks the number of redirects and identifies whether the domain changes during redirection. Multiple redirects or domain mismatches are

common phishing techniques, and this feature helps uncover hidden malicious behavior.

1.3.8. Subdomain & Homograph Attack Detection

The system examines subdomains to identify excessive or misleading domain structures that mimic legitimate websites. It also detects homograph attacks, where attackers use visually similar characters (such as “0” instead of “o”) to deceive users. These techniques are commonly used in phishing, and detecting them improves system reliability.

1.3.9. Threat Scoring Mechanism

The system assigns a threat score to each URL based on multiple factors, including internal analysis, API results, and security checks. Based on this score, the URL is classified as safe, suspicious, or phishing. This graded evaluation provides users with a clearer understanding of the risk level rather than a simple binary result.

1.3.10. Database Storage and Admin Panel

The system stores all scanned URLs, results, and threat scores in a database. Admins can view the full scan history, filter or sort results by status (Safe, Suspicious, Phishing), manually label or correct URL classifications, and download reports in CSV or PDF format. This enables efficient monitoring, manual corrections, and better tracking of phishing threats over time.

1.3.11. Interactive Web Interface

The system includes a user-friendly web interface developed using Flask, allowing users to easily input URLs and view results. The interface displays detailed analysis such as IP information, WHOIS data, SSL status, and threat score in an organized manner, making the system accessible to both technical and non-technical users.

CHAPTER 2

LITERATURE REVIEW

[1] Ume Zara, Kashif Ayyub, Hikmat Ullah Khan, Ali Daud, Tariq Alsahfi, Saima Gulzar Ahmad, “Phishing Website Detection Using Deep Learning Models,” IEEE Access, Vol. 12(2), October 2024

The study explores the use of advanced machine learning and deep learning techniques to detect phishing websites effectively, as presented in the paper by Ume Zara et al. published in IEEE Access (2024). It focuses on addressing the increasing threat of phishing attacks, where fraudulent websites mimic legitimate ones to steal sensitive user information such as login credentials and financial data. The authors develop a detection system that extracts important URL-based and security-related features, including domain properties and hyperlink behavior, and applies feature selection techniques like Principal Component Analysis (PCA) and Information Gain to improve efficiency. The study compares various models, including traditional algorithms like Support Vector Machine (SVM) and Decision Tree, along with deep learning models such as Long Short-Term Memory (LSTM) and Recurrent Neural Networks (RNN), which are capable of capturing complex patterns in data. Ensemble methods like Random Forest and XGBoost are also used to enhance prediction accuracy. The results show that deep learning and hybrid approaches outperform conventional methods in terms of accuracy, precision, and reliability. Overall, the study demonstrates that combining optimized feature selection with deep learning models provides a robust and scalable solution for real-time phishing detection, helping improve cybersecurity and protect users from online threats. However, the approach requires significant computational resources and large datasets for effective training. Overall, this research emphasizes the importance of intelligent, automated detection systems in modern cybersecurity, contributing to more reliable and scalable solutions for protecting users against phishing attacks.

[2] A. K. Jain and B. B. Gupta, “PHISH-SAFE: URL Features-Based Phishing Detection System Using Machine Learning,” in Cyber Security (Advances in Intelligent Systems and Computing), Vol. 729, Springer, pp. 467–474, April 2018.

The study explores a machine learning-based approach for detecting phishing websites through URL feature analysis, as presented by A. K. Jain and B. B. Gupta in *PHISH-SAFE* published in Cyber Security (Advances in Intelligent Systems and Computing) (2018). It focuses on identifying phishing attacks by extracting significant features directly from URLs, such as length, presence of special characters, and domain-related properties, without relying on third-party services. The proposed system applies machine learning algorithms to classify websites as legitimate or phishing, improving detection speed and efficiency. The study demonstrates that URL-based feature extraction combined with classification models can achieve reliable accuracy while maintaining low computational cost. It also emphasizes that this approach is lightweight and suitable for real-time detection systems. However, the method may struggle with highly sophisticated phishing techniques that closely mimic legitimate URLs. The authors also compare different classifiers to evaluate their performance in terms of accuracy and detection rate. The system is designed to minimize false positives, ensuring legitimate websites are not incorrectly flagged. It highlights the importance of selecting relevant features to improve model performance. The approach reduces dependency on blacklist-based methods, which may not detect newly created phishing sites. The study also suggests that integrating additional features could further enhance detection capability. Experimental results indicate that the model performs efficiently in real-time scenarios. Overall, the research supports the development of fast, lightweight, and effective phishing detection systems for practical cybersecurity applications.

[3] J. K. Lee, Y. Chang, H. Y. Kwon, and B. Kim , “Reconciliation of Privacy with Preventive Cybersecurity: The Bright Internet Approach,” *Information Systems Frontiers*, Vol. 22, No. 1, pp. 45–57, February 2020.

The study explores the balance between user privacy and proactive cybersecurity measures, as presented by J. K. Lee, Y. Chang, H. Y. Kwon, and B. Kim in “*Reconciliation of Privacy with Preventive Cybersecurity: The Bright Internet Approach*” published in *Information Systems Frontiers* (2020). It introduces the concept of the Bright Internet, a framework designed to create a safer online environment by embedding accountability and traceability into internet usage while still preserving user privacy. The study discusses how traditional cybersecurity measures often conflict with privacy protection, and proposes a structured approach that enables identification of malicious activities without exposing sensitive personal data. The Bright Internet model incorporates mechanisms such as identity management, secure authentication, and controlled access to ensure responsible online behavior. It emphasizes the importance of regulatory policies and technological solutions working together to prevent cyber threats such as phishing, malware, and cyber fraud. The authors highlight that improving transparency and accountability can significantly reduce anonymous malicious activities on the internet. Additionally, the study discusses challenges such as dataset imbalance, evolving phishing techniques, and high computational requirements. However, the approach also raises challenges related to implementation complexity and potential resistance due to privacy concerns. The study suggests that achieving a balance between privacy and security is essential for the future of digital systems. It also underlines the need for global cooperation and standardized frameworks to successfully implement such models. Overall, the research contributes to the development of secure, privacy-aware cybersecurity strategies for modern internet ecosystems.

[4] S. Asiri, Y. Xiao, S. Alzahrani, S. Li, and T. Li, “A Survey of Intelligent Detection Designs of HTML URL Phishing Attacks,” IEEE Access, Vol. 11, pp. 6421–6443, January 2023.

This survey analyzes various intelligent techniques for detecting HTML and URL-based phishing attacks, as presented by S. Asiri, Y. Xiao, S. Alzahrani, S. Li, and T. Li in IEEE Access (2023). It provides a comprehensive survey of existing phishing detection methods, including traditional machine learning, deep learning, and hybrid approaches used to identify malicious websites. The study analyzes different types of features such as URL structure, HTML content, JavaScript behavior, and visual similarity to legitimate websites. It highlights how deep learning models, including Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN), are increasingly effective in capturing complex patterns in phishing attacks. The paper also compares various detection systems based on performance metrics like accuracy, precision, and recall. It emphasizes that combining multiple feature types can significantly improve detection performance. Additionally, the study discusses challenges such as dataset imbalance, evolving phishing techniques, and high computational requirements. The authors point out the limitations of existing systems in handling zero-day phishing attacks. The survey suggests that future research should focus on lightweight and real-time detection models. It also highlights the importance of integrating intelligent detection systems into web browsers and security tools. Overall, the study provides valuable insights into current trends and future directions in phishing detection research. Furthermore, it emphasizes the need for continuous dataset updates to keep up with emerging threats. The study also encourages the use of hybrid models that combine speed and accuracy. It concludes that adaptive and scalable solutions are essential for combating modern phishing attacks effectively.

[5] N. A. Azeez, S. Misra, I. A. Margaret, L. Fernandez-Sanz, and S. M. Abdulhamid, “Adopting Automated Whitelist Approach for Detecting Phishing Attacks,” *Computers & Security*, Vol. 108, September 2021.

This study demonstrates an automated whitelist-based approach for detecting phishing attacks, as presented by N. A. Azeez, S. Misra, I. A. Margaret, L. Fernandez-Sanz, and S. M. Abdulhamid in *Computers & Security* (2021). It focuses on improving phishing detection by maintaining a list of trusted and verified websites, allowing the system to quickly identify legitimate sources and flag unknown or suspicious URLs. The proposed approach automates the process of updating and managing the whitelist, reducing manual effort and enhancing system efficiency. The study highlights that this method provides faster detection compared to blacklist-based systems, as it directly validates trusted domains instead of searching for known malicious ones. It also minimizes false positives by ensuring that only verified websites are marked as safe. The authors discuss how the system can be integrated with existing security frameworks to strengthen overall protection. However, the approach may face limitations when dealing with newly emerging legitimate websites that are not yet included in the whitelist. The study also emphasizes the need for continuous updating of the whitelist database to maintain accuracy. Experimental results show that the method is effective in real-time phishing detection scenarios. Furthermore, the research suggests combining whitelist approaches with machine learning techniques for improved performance. It concludes that automated whitelist systems can play a significant role in enhancing cybersecurity by providing reliable and efficient phishing detection.

CHAPTER 3

SYSTEM ANALYSIS

3.1.EXISTING SYSTEM

The existing system for phishing website detection, as proposed by Ume Zara et al. (2024), is primarily based on deep learning algorithms. The system utilizes advanced neural network models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) to automatically learn patterns from large datasets of URLs and webpage content. These models are capable of capturing complex relationships between various features, including URL structure, HTML content, and behavioral patterns of websites. Unlike traditional machine learning methods that rely on manually engineered features, deep learning models extract features automatically through multiple hidden layers, improving detection accuracy.

In this system, a large dataset containing both legitimate and phishing URLs is used for training the model. The data undergoes preprocessing and feature extraction before being fed into the deep learning architecture. During training, the model learns to differentiate between safe and malicious websites by identifying subtle variations and hidden patterns. Once trained, the system can classify incoming URLs as phishing or legitimate. This approach provides high accuracy and adaptability in detecting sophisticated phishing attacks, making it a significant advancement over rule-based and traditional machine learning techniques.

3.1.1. Disadvantages

- Requires a large amount of labeled training data, which is difficult and time-consuming to collect.
- High computational requirements, often needing powerful hardware such as GPUs.

- Complex model architecture increases implementation and maintenance difficulty.
- Lacks integration with real-time threat intelligence sources like VirusTotal.
- Unable to effectively detect newly emerging phishing websites not present in training data.
- Possibility of false negatives when encountering unseen or zero-day attacks.
- Not suitable for lightweight or low-resource environments.

3.1.2. Scalability Challenges

- Training deep learning models on large datasets requires significant time and computational resources.
- Difficult to update the model frequently with new phishing data.
- Performance may degrade as the volume of incoming URLs increases.
- High memory and processing requirements limit deployment on web-based or real-time systems.
- Not efficient for large-scale, real-time URL analysis applications.
- Scaling the system for multiple users or high traffic environments can be costly and complex.

3.2 PROPOSED SYSTEM

The proposed system, AI-Powered URL Security and Phishing Detection System with Threat Intelligence Integration, is designed to overcome the limitations of traditional phishing detection methods by combining intelligent URL analysis with real-time threat intelligence services. Unlike the existing system, which relies solely on deep learning models, the proposed system adopts a hybrid approach that integrates rule-based analysis, machine learning concepts, and external security APIs. This combination ensures

improved detection accuracy, faster response time, and better adaptability to evolving cyber threats.

In the proposed system, the detection process begins when a user inputs a URL through a web-based interface. The system first normalizes the URL and extracts multiple features such as URL length, presence of suspicious keywords, number of subdomains, and use of HTTPS protocol. These features are evaluated using a scoring-based mechanism inspired by machine learning techniques, which can be further enhanced using models like XGBoost. This internal analysis forms the first layer of detection and quickly identifies suspicious patterns commonly used in phishing attacks.

To strengthen detection capabilities, the system integrates external threat intelligence services such as VirusTotal and Google Safe Browsing. These APIs provide real-time information about known malicious URLs, phishing domains, and malware distribution sites. By combining internal analysis with global threat intelligence, the system can detect both known and unknown threats more effectively. This integration ensures that the system remains updated with the latest cybersecurity threats and significantly reduces the chances of false negatives.

In addition to API integration, the proposed system incorporates multiple security checks such as IP address analysis, WHOIS lookup, and SSL certificate validation. It also performs redirect analysis, subdomain inspection, and homograph attack detection to identify deceptive techniques used by attackers. These features provide a multi-layered security approach, where each component contributes to evaluating the legitimacy of a URL. The system calculates a threat score based on all these factors and classifies the URL as safe, suspicious, or phishing, providing users with a detailed risk assessment.

The system is implemented using Python with the Flask framework, ensuring a lightweight and efficient architecture suitable for real-time applications. It also includes a database to store scanned URLs and results,

enabling tracking and administrative analysis. Compared to deep learning-based systems, the proposed approach requires fewer computational resources, making it more scalable and easier to deploy. Overall, the system provides a practical, efficient, and robust solution for detecting phishing websites while ensuring fast performance and user-friendly interaction.

3.2.1. Advantages

- Combines multiple detection techniques (AI + rule-based + APIs) for higher accuracy.
- Integrates real-time threat intelligence using VirusTotal and Google Safe Browsing.
- Faster response time compared to deep learning-based systems.
- Requires less computational power and no need for high-end GPUs.
- Detects both known and unknown phishing attacks effectively.
- Provides detailed analysis including IP, WHOIS, SSL, and redirect information.
- User-friendly web interface for easy interaction.
- Stores scan history for tracking and analysis.

3.2.2. Overcoming Existing Limitations

- Eliminates dependency on large training datasets by using hybrid analysis.
- Reduces computational cost compared to deep learning models.
- Improves detection of new and unknown threats using real-time APIs.
- Minimizes false negatives through multi-layered security checks.
- Enhances scalability with lightweight and efficient architecture.
- Enables real-time detection without heavy processing delays.
- Supports continuous updates through external threat intelligence sources.

CHAPTER 4

SYSTEM REQUIREMENTS

The AI-Powered URL Security and Phishing Detection System is designed to analyze and classify URLs in real time to determine whether they are safe, suspicious, or phishing. The system accepts user input through a web interface, processes the URL by extracting various features, and performs multiple security checks such as IP analysis, domain information retrieval, SSL certificate validation, and redirect detection. It also integrates external threat intelligence services like VirusTotal and Google Safe Browsing to enhance detection accuracy. Based on the combined analysis, the system generates a threat score and displays the result along with detailed security information. Additionally, the system stores scan results in a database for future reference and administrative monitoring.

4.1 HARDWARE REQUIREMENTS

- **Processor:** Intel Core i5 or above
- **Memory:** Minimum 8 GB RAM
- **Storage:** At least 5GB free disk space
- **Network:** Stable internet connection
- **Device:** Desktop or Laptop

4.2. SOFTWARE REQUIREMENTS

- **Development Environment:** Visual Studio Code / PyCharm
- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python 3.10 or above
- **Framework:** Flask
- **Database:** SQLite
- **Libraries:** requests, whois, ssl, socket, re, urllib
- **Web Technologies:** HTML, CSS, JavaScript

- **APIs:** VirusTotal, Google Safe Browsing
- **Browser:** Google Chrome / Edge / Firefox

4.3. HARDWARE DESCRIPTION

4.3.1. Processor

The processor is the main component of a computer that performs all calculations and executes instructions. In this project, the processor handles URL analysis, feature extraction, API requests, and overall system execution. It ensures that the system processes user inputs quickly and performs real-time phishing detection efficiently.



Figure 4.1 intel i5 Processor

4.3.2. Memory

The system requires min 8 GB DDR4 RAM to handle real-time operations efficiently. In this project, RAM is used to hold the application data, process URL inputs, and manage API responses during execution. Sufficient RAM ensures smooth performance without lag while handling multiple operations simultaneously.

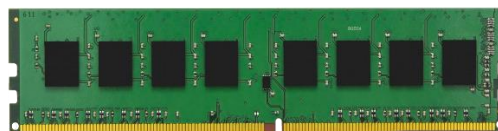


Figure 4.2 8GB DDR4 RAM

4.3.3. Storage

A minimum of 512 GB SSD storage is necessary for storing the application files, database (SQLite), scanned URL history, and results. It ensures that all records are maintained for future reference and administrative analysis.



Figure 4.3 512 GB SSD

4.4 SOFTWARE DESCRIPTION

4.4.1. Development Environment

Visual Studio Code / PyCharm

The system is developed and executed using Visual Studio Code and PyCharm, which are powerful integrated development environments (IDEs) offering features such as debugging, code completion, and version control integration.

4.4.2. Operating System

Windows / Linux / macOS

The operating system is the platform that manages hardware and software resources. In this project, it provides the environment to run Python, Flask, and other required libraries. It ensures smooth execution of the application and supports system-level operations such as networking and file management.

4.4.3. Programming Language

Python 3.10

Python is the main programming language used to develop the system. It is responsible for implementing URL analysis, feature extraction, API

integration, and backend logic. In this project, Python handles all core functionalities including threat scoring, data processing, and communication between modules.

4.4.4. Libraries

4.4.4.1. Requests

The requests library is used to send HTTP and HTTPS requests to external services such as VirusTotal and Google Safe Browsing APIs. It allows the system to transmit user-input URLs and receive real-time responses about their safety status. The responses include threat intelligence data such as malware reports and phishing indicators. It supports handling JSON data, making it easy to process API results. The library also manages headers, parameters, and authentication if required. It ensures reliable communication with external servers. Error handling features help manage failed or delayed requests. Overall, it plays a key role in integrating real-time security checks into the system.

4.4.4.2. WHOIS

The whois library is used to retrieve domain registration details from WHOIS databases. It provides information such as domain registrar, creation date, and expiration date. These details help in identifying suspicious domains, especially newly registered ones. Phishing websites often use short-lived domains to avoid detection. The library helps assess the credibility and age of a domain. It also supports identifying unknown or hidden ownership details. This information is useful in detecting fraudulent websites. Overall, it strengthens domain-level analysis in phishing detection.

4.4.4.3. SSL

The ssl library is used to establish secure connections and verify SSL certificates of websites. It checks whether a website uses HTTPS and has a valid certificate. The module extracts details such as certificate issuer and validity period. Expired or untrusted certificates indicate potential security risks. It helps ensure that the connection between user and server is encrypted. The library

also supports certificate fingerprint verification. It plays an important role in detecting insecure or fake websites. Overall, it enhances security validation in the system

4.4.4.4. Socket

The socket library is used for low-level network communication and domain resolution. It converts domain names into IP addresses through DNS lookup. This helps identify the server hosting the website. It supports establishing connections for SSL verification. The library also enables retrieval of server-related network information. It is useful for performing IP-based analysis. Suspicious IP behavior can indicate phishing activity. Overall, it supports deeper network-level inspection in the system.

4.4.4.5. RE (Regular Expressions)

The re library is used for pattern matching and string analysis in URLs. It helps detect suspicious patterns such as IP-based URLs and unusual characters. It identifies phishing-related keywords like “login”, “verify”, and “secure”. Regular expressions allow flexible and efficient pattern detection. It can validate URL structure and detect anomalies. This helps in identifying malicious URL formats. The library improves feature extraction accuracy. Overall, it plays a vital role in heuristic analysis.

4.4.4.5. URLlib.parse

The urllib.parse module is used to parse and manipulate URLs effectively. It extracts components such as protocol, domain name, and path. This helps in normalizing URLs before analysis. It ensures consistency in URL formatting. The module supports splitting and reconstructing URLs. It improves accuracy in feature extraction. It also helps detect suspicious URL structures. Overall, it is essential for preprocessing and structural analysis of URLs.

4.4.5. Web Technologies

HTML, CSS, JavaScript

The HTML is used to design the structure of web pages. In this project, it creates the user interface where users input URLs and view results. It defines elements such as forms, buttons, and display sections. CSS is used to style and design the web interface. In this project, it enhances the appearance of the application by adding layouts, colours, and visual effects, improving user experience and readability. JavaScript is used to add interactivity to web pages. In this project, it helps in dynamic UI updates, form validation, and improving user interaction with the system.

4.4.6. APIs

VirusTotal API

This API provides real-time threat intelligence by checking URLs against multiple antivirus engines. In this project, it is used to identify whether a URL is associated with malware or phishing activities, improving detection accuracy.

Google Safe Browsing API

This API helps detect unsafe websites related to phishing, malware, and social engineering. In this project, it provides additional verification by checking URLs against Google's continuously updated threat database.

4.4.7. Web Browser

Chrome / Firefox / Edge

A web browser is used to access and interact with the application. In this project, users open the Flask web interface through a browser to input URLs and view analysis results. It acts as the front-end platform for user interaction.

CHAPTER 5

SYSTEM DESIGN

5.1. SYSTEM ARCHITECTURE

The AI-Powered URL Security and Phishing Detection System follows a modular and layered architecture designed to provide efficient, real-time analysis of URLs. The system integrates user interaction, feature extraction, security analysis, machine learning-based scoring, and external threat intelligence services into a unified framework. Each module performs a specific function, and together they ensure accurate detection of phishing and malicious websites.

The architecture begins with the user interface, where users input a URL through a web browser. The request is then handled by the Flask-based backend server, which acts as the core processing unit. The system processes the URL through multiple stages, including normalization, feature extraction, internal analysis, and external API verification. The results from all modules are combined to generate a final threat score and classification, which is displayed to the user along with detailed information.

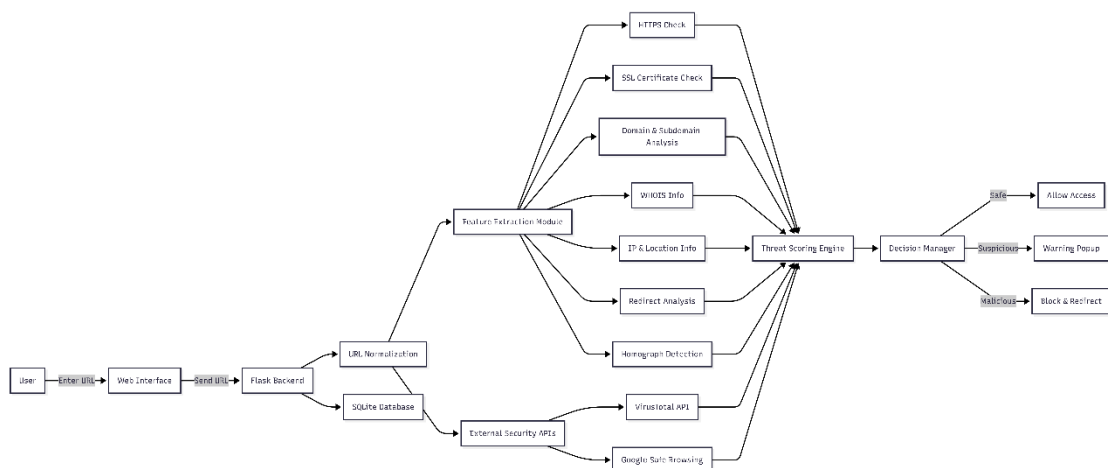


Figure 5.1 Architecture Diagram

5.2. DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) illustrates how data moves through the AI-Powered URL Security and Phishing Detection System. The system is divided into multiple modules such as URL normalization, feature extraction, internal analysis, external API checks, and security analysis. The processed data flows through these modules to generate a threat score and classification, which is then stored in the database and displayed to the user. The admin can access stored data for monitoring and analysis.

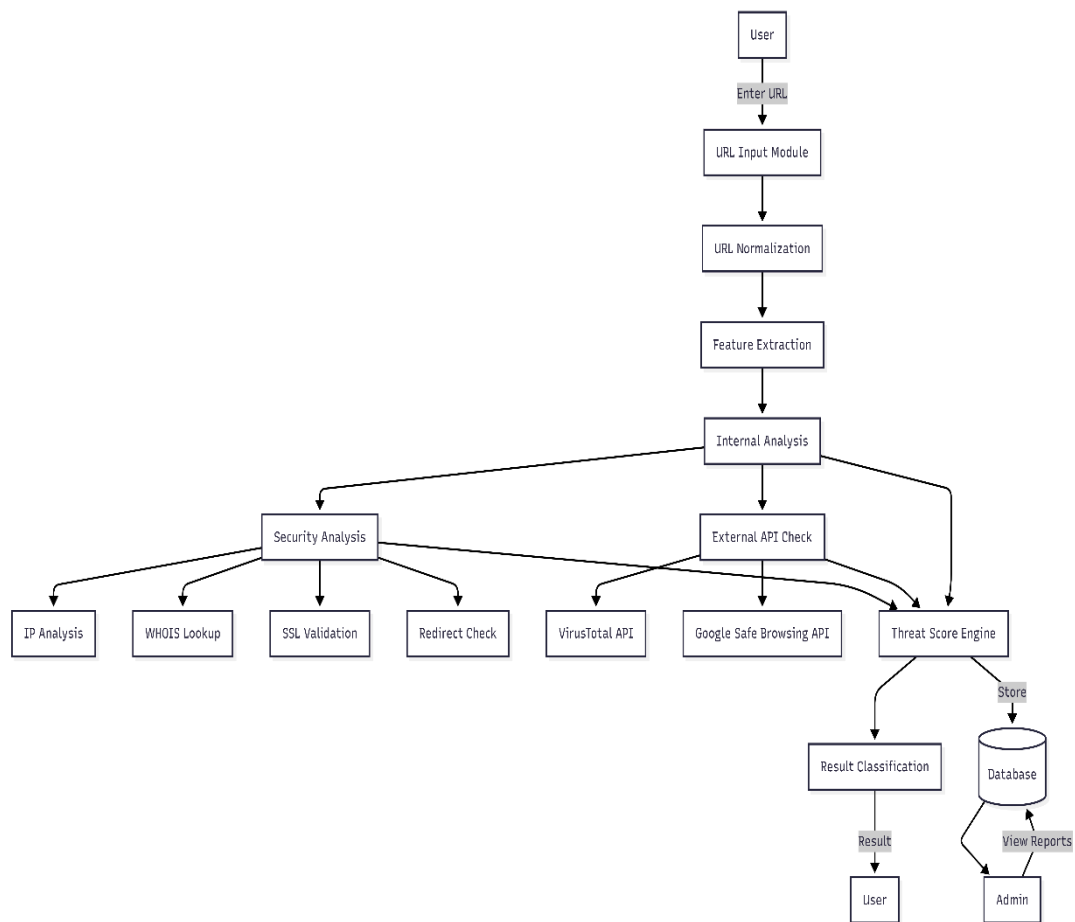


Figure 5.2 Flow Diagram

The Figure 5.2 Data Flow Diagram illustrates the AI-Powered URL Security and Phishing Detection Process

- **User Input (URL Submission):** The process begins when the user enters a URL into the system through the web interface for analysis.
- **URL Normalization:** The input URL is standardized by adding missing protocols (HTTP/HTTPS), removing unnecessary characters, and formatting it for consistent processing.
- **Feature Extraction:** The system extracts important features such as URL length, presence of suspicious keywords, number of subdomains, and HTTPS usage. These features are essential for identifying phishing patterns.
- **Internal Analysis (Rule-Based / AI-Based):** The extracted features are analyzed using rule-based techniques and AI models such as XGBoost to detect suspicious patterns and assign an initial threat score.
- **External Threat Intelligence Integration:** The system checks the URL using external services like VirusTotal and Google Safe Browsing to identify known malicious or phishing websites.
- **Security Analysis Module:** The system performs multiple security checks including IP address analysis, WHOIS lookup, SSL certificate validation, and redirect analysis to evaluate the legitimacy of the website.
- **Threat Score Engine:** The results from internal analysis, external APIs, and security checks are combined to calculate a final threat score representing the risk level of the URL.
- **Result Display:** The final result along with detailed analysis is displayed to the user through the web interface.
- **Admin Monitoring:** The admin can access stored scan data from the database to monitor system performance and analyze past results.

5.3. USE CASE DIAGRAM

A Use Case Diagram is a type of UML diagram that represents how users interact with a system and helps in understanding the system's functionality from an external perspective. It mainly focuses on the functional requirements and shows what the system does rather than how it does it. The diagram consists of key components such as actors, use cases, and the system boundary. Actors are external entities like users or administrators who interact with the system, while use cases represent the various actions or services provided by the system. The relationships between actors and use cases are shown using lines, indicating interaction or communication. This diagram makes it easy to identify who can perform specific operations within the system. It is widely used during the requirement analysis phase to gather and organize system functionalities.

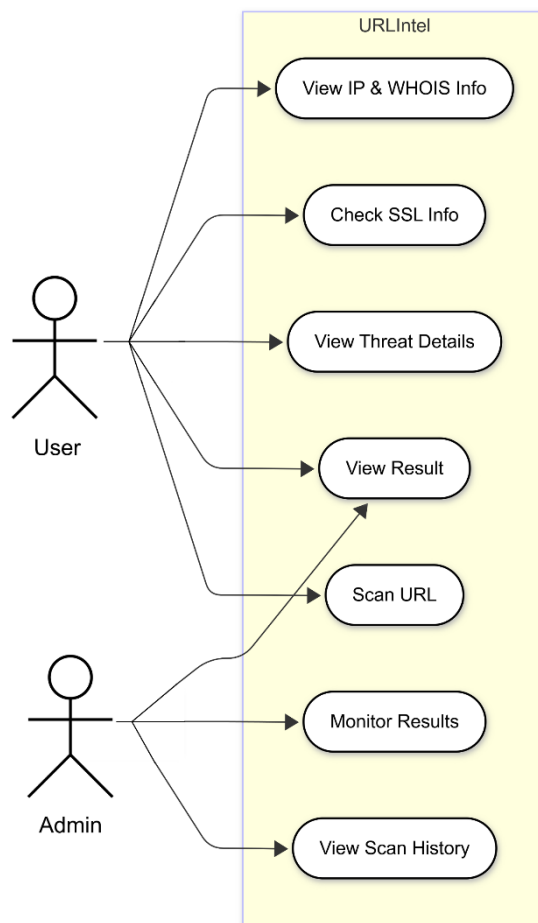


Figure 5.3 Use Case Diagram

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 LIST OF MODULES

- User Interface Module
- URL Processing & Normalization Module
- Feature Extraction & Heuristic Analysis Module
- External Threat Intelligence Module
- Network & Security Analysis Module
- Domain & IP Intelligence Module
- Data Storage & Admin Module

6.2 MODULE DESCRIPTION

6.2.1. User Interface Module

This module acts as the front-end layer of the system, enabling seamless interaction between the user and the application. It is implemented using Flask routes such as /, /scan, and /admin, along with HTML templates like index.html, result.html, and admin.html. Users can easily input a URL through a form, and the system processes the request and displays detailed results including threat score, classification (Safe/Suspicious/Phishing), SSL status, and IP-related information. The interface is designed to be intuitive, responsive, and user-friendly, ensuring accessibility even for non-technical users. It has 3D interactive patterns for best use experience. It also provides clear visual feedback, structured outputs, and proper error handling messages when invalid URLs are entered. Additionally, the UI can be enhanced with charts or color indicators to represent threat levels visually, improving user understanding. The module ensures smooth navigation between pages and efficient communication with backend services.

6.2.2. URL Processing & Normalization Module

This module ensures that all input URLs are standardized before analysis. It handles tasks such as adding missing protocols (e.g., https://), removing unnecessary prefixes like www, and restructuring URLs using functions like `urlparse` and `urlunparse`. It also removes trailing slashes, resolves relative paths, and ensures consistent formatting across all inputs. Normalization eliminates inconsistencies and prevents duplicate analysis of the same website in different formats. It also improves caching efficiency if results are stored. Proper preprocessing ensures that extracted features are accurate and consistent, which directly enhances model performance. This module plays a crucial role in maintaining data integrity and preparing inputs for downstream analysis modules.

6.2.3. Feature Extraction & Heuristic Analysis Module

This is the core analytical component of the system where the main phishing detection logic is applied. It extracts multiple features from the URL, such as length, number of dots, use of IP address instead of domain name, presence of suspicious keywords (e.g., “login”, “verify”, “bank”), special characters, and abnormal subdomain patterns. It also evaluates URL entropy and checks for encoding tricks used by attackers. In addition to feature extraction, it applies a heuristic scoring mechanism where each feature contributes to an overall threat score. The module classifies URLs into Safe, Suspicious, or Phishing categories based on predefined thresholds. It provides fast preliminary detection without relying on external APIs, making it efficient for real-time usage. This module can also be extended by integrating machine learning models for improved accuracy.

6.2.4. External Threat Intelligence Module

This module enhances detection accuracy by integrating third-party cybersecurity services. It uses APIs such as VirusTotal to analyze URLs against multiple antivirus engines and Google Safe Browsing to identify known malicious websites. These services provide continuously updated threat

intelligence from global sources. The module fetches and processes API responses, extracts relevant risk indicators, and integrates them into the system's final decision. It helps detect zero-day and recently reported phishing sites that may not be identified by local analysis alone. However, it also considers API limitations such as rate limits and response time. Overall, this module significantly improves reliability and ensures up-to-date threat detection.

6.2.5. Network & Security Analysis Module

This module performs advanced technical checks on the URL to uncover hidden threats. It validates SSL certificates to ensure secure communication and extracts certificate details such as issuer, validity period, and encryption type. It also analyzes HTTP response headers, checks for redirection chains, and identifies suspicious forwarding behavior. DNS resolution is performed to obtain the IP address and verify domain legitimacy. Additionally, the module detects homograph attacks by comparing visually similar characters in domain names. These checks help identify sophisticated phishing techniques that may bypass basic detection methods. This module strengthens the system by adding a deeper layer of security analysis.

6.2.6. Domain & IP Intelligence Module

This module gathers detailed metadata about the domain and its hosting environment. It performs WHOIS lookup to obtain information such as domain registration date, expiry date, and registrar details. Newly registered domains are often flagged as suspicious. The module also retrieves IP geolocation data, including country, city, and ISP information, using external APIs. This helps detect unusual hosting locations commonly used in phishing campaigns. It can also analyze domain reputation and historical usage patterns. By combining domain and IP intelligence, the system gains better context for decision-making. This module plays a key role in identifying suspicious infrastructure behind phishing attacks..

6.2.7. Data Storage Module

This module is responsible for storing all the data generated by the system in a structured and persistent manner using an SQLite database. Every time a user scans a URL, the system saves important details such as the input URL, extracted features, threat score, classification result (Safe/Suspicious/Malicious), SSL status, IP address, and timestamp. This allows the system to maintain a complete history of all analyzed URLs for future reference. SQLite is chosen because it is lightweight, fast, and does not require a separate server, making it ideal for small to medium-scale applications. The module also ensures efficient data retrieval for displaying past results and avoids redundant analysis by checking previously scanned URLs. Proper database schema design is implemented to organize data into tables, ensuring consistency and easy querying. Additionally, this module supports scalability by allowing future integration with larger databases if needed. Overall, it plays a crucial role in maintaining data integrity, enabling tracking, and supporting system improvements through stored information.

6.2.8. Admin Module

This module provides administrative control and monitoring capabilities for the system through a dedicated admin panel (e.g., /admin). It allows the administrator to view all previously analyzed URLs along with their results, threat scores, and timestamps in a structured format. One of the key features of this module is the ability to manually update or override classifications, enabling the admin to mark any URL as Safe, Suspicious, or Malicious based on expert judgment. This helps improve system accuracy and supports continuous learning. The module also includes functionality to download the complete scan history in formats such as CSV or PDF for reporting, auditing, or further analysis. The admin panel enhances transparency and gives full control over stored data, making it easier to monitor system performance and detect patterns in phishing attempts. Overall, this module is essential for managing, reviewing, and maintaining the effectiveness of the phishing detection system.

CHAPTER 7

RESULTS AND DISCUSSION

The developed system, AI-Powered URL Security and Phishing Detection System (URLIntel), was successfully tested with multiple types of URLs including safe, suspicious, and phishing websites. The system analyzes the given URL in real-time and generates a threat score (0–100) along with a classification: Safe, Suspicious, or Phishing.

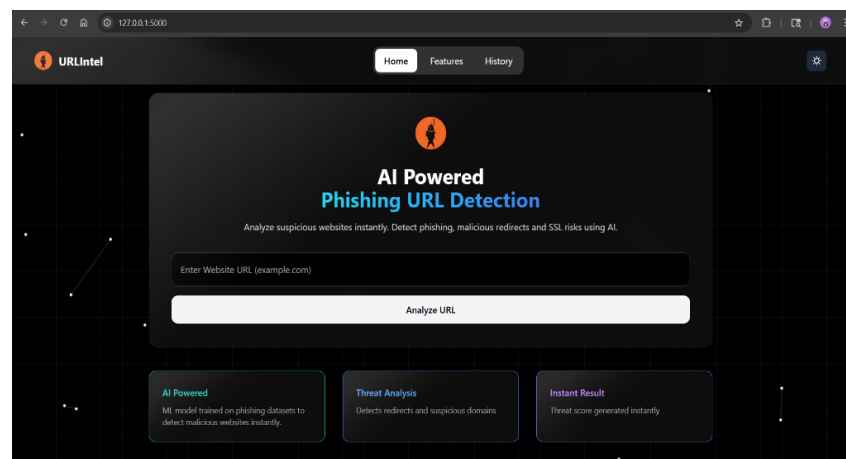


Figure 7.1 Entering the URL

The index page of the URLIntel system serves as the main entry point where users can input a website URL for analysis. It features a clean and modern dark-themed interface with a central input field labeled “Enter Website URL” and an “Analyze URL” button for initiating the scan. The page also highlights key features such as AI-powered detection, threat analysis, and instant results, helping users understand the system’s capabilities. The design is user-friendly and intuitive, allowing even non-technical users to easily submit URLs and begin the security analysis process efficiently.

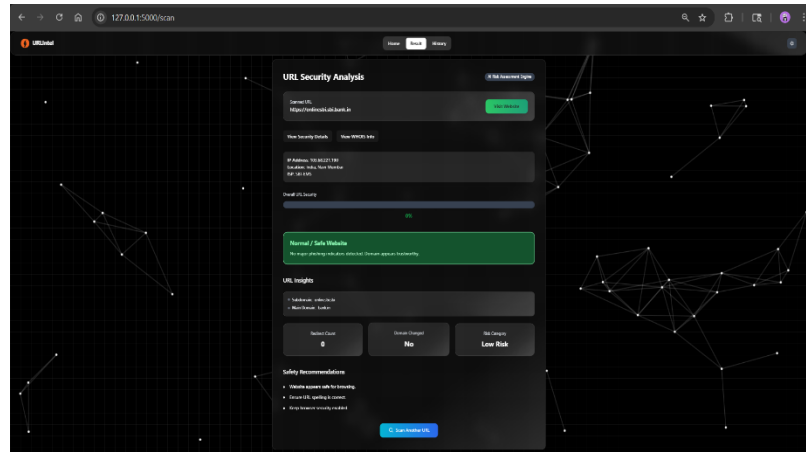


Figure 7.2 Safe Website Result

The result page for a safe website displays a well-structured and visually clear interface indicating a low threat score (0%) with a green progress bar, representing a secure and trustworthy domain. The system identifies no suspicious behavior such as redirects or domain mismatches, and the classification “Normal / Safe Website” is prominently shown. Additional details like IP address, location, ISP, and domain structure are provided, reinforcing transparency.

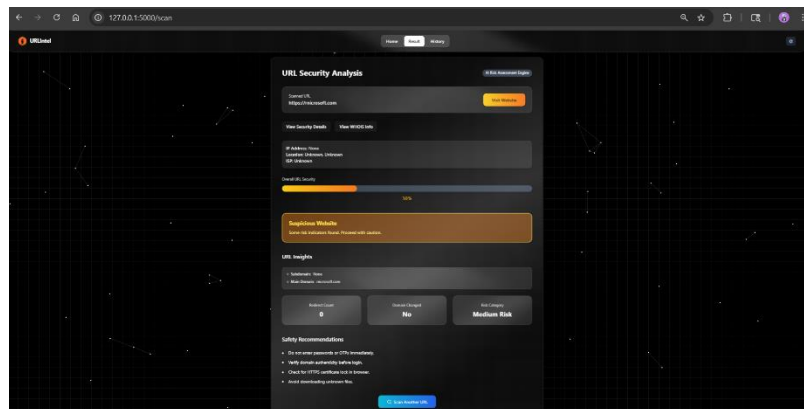


Figure 7.3 Suspicious Website Result

In the case of a suspicious website, the result page highlights a moderate threat score (around 30%) using a yellow/orange progress bar, indicating potential risk. The system labels the site as “Suspicious Website” and identifies minor irregularities such as incomplete IP data or subtle domain issues. While

no major phishing indicators are detected, the system provides cautionary recommendations, encouraging users to verify authenticity before interacting. This demonstrates the system’s ability to detect early warning signs without misclassifying the site as fully malicious.

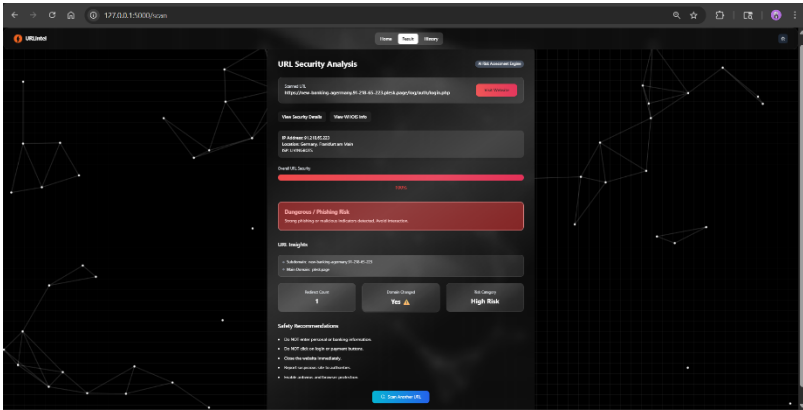


Figure 7.4 Phishing Website Result

For a phishing or dangerous website, the result page clearly emphasizes a high threat score (100%) with a red progress bar, signaling severe risk. The classification “Dangerous / Phishing Risk” is prominently displayed, along with critical indicators such as suspicious subdomains, redirect activity, and domain mismatches. The system also provides detailed IP information and flags high-risk behavior, helping users understand the threat. Strong safety recommendations are included, advising users to avoid interaction detecting and warning against malicious websites.

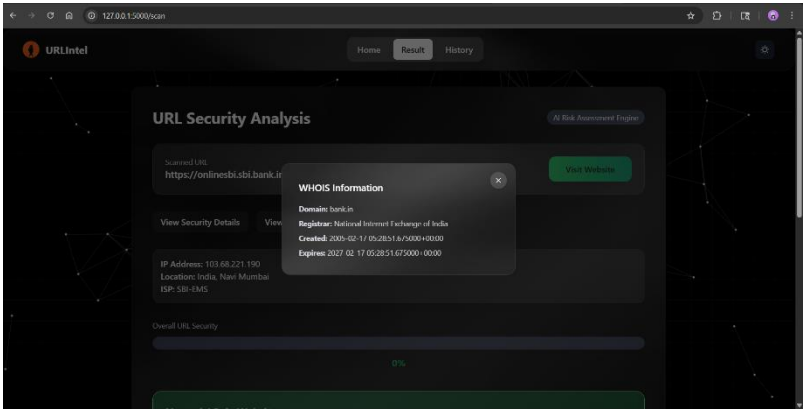


Figure 7.5 WHOIS Information View

The WHOIS information pop-up provides detailed domain registration data, helping users verify the authenticity and credibility of the website. It displays key details such as the domain name, registrar (National Internet Exchange of India), creation date, and expiration date, which are important indicators of domain legitimacy. In this case, the domain shows a long registration history and valid future expiry, suggesting it is a well-established and trusted domain. By presenting this information in a clear and structured format, the system enables users to assess whether a domain is newly created or suspicious, thereby improving decision-making and enhancing overall security awareness.

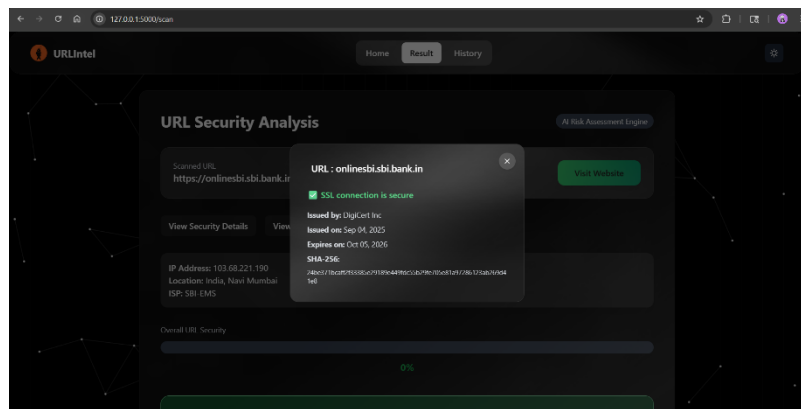


Figure 7.6 SSL Certificate Details

The SSL details pop-up provides a focused and informative view of the website's security certificate, enhancing transparency in the analysis process. It clearly indicates that the SSL connection is secure using a green status indicator, building user confidence. The panel displays key certificate information such as the issuing authority (DigiCert Inc), issue date, expiry date, and SHA-256 fingerprint, which are essential for verifying the authenticity and validity of the website. This feature allows users to inspect encryption details without cluttering the main result page, making the system both technically informative and user-friendly.

CHAPTER 8

SAMPLE CODE

```
from flask import Flask, render_template, request
import sqlite3
import re
import ssl
import whois
import socket
import requests
from flask import request
from urllib.parse import urlparse, urlunparse
from datetime import datetime
app = Flask(__name__)
def init_db():
    conn = sqlite3.connect("scans.db")
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS scans (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            url TEXT,
            result TEXT,
            threat_score INTEGER
        )
    """)
    conn.commit()
    conn.close()
init_db()
TRUSTED_DOMAINS = [
    "sbi.co.in",
```

```

"onlinesbi.sbi.bank.in",
"icicibank.com",
"hdfcbank.com",
"axisbank.com",
"kotak.com",
"pnbindia.in"
]
PHISH_DOMAINS = [
    "raiodesolbrilhante1.com.br",
    "malicious-example.com",
    "https://joxoh.life/onlinebanking.libertyfcu",
    "https://raiodesolbrilhante1.com.br/plala/jpn/webmail/index.php",
    "https://kleinanzeigen.mx51088081.com/receive/9653421"
]
def normalize_url(url):
    url = url.strip()
    if not url.startswith("http://") and not url.startswith("https://"):
        url = "https://" + url
    parsed = urlparse(url)
    scheme = parsed.scheme # ↩ keep original http/https
    netloc = parsed.netloc.lower().replace("www.", "")
    path = parsed.path.rstrip("/")
    return urlunparse((scheme, netloc, path, "", "", ""))
def check_virustotal(url):
    API_KEY = ""
    headers = {"x-apikey": API_KEY}
    try:
        r = requests.post(
            "https://www.virustotal.com/api/v3/urls",
            headers=headers,
            data={"url": url}

```

```

    )
    analysis_id = r.json()["data"]["id"]
    report = requests.get(
        f"https://www.virustotal.com/api/v3/analyses/{analysis_id}",
        headers=headers
    )
    return report.json()["data"]["attributes"]["stats"]
except:
    return None
def check_google_safe(url):
    API_KEY = ""
    body = {
        "client": {"clientId": "url-intel", "clientVersion": "1.0"},
        "threatInfo": {
            "threatTypes": ["MALWARE", "SOCIAL_ENGINEERING"],
            "platformTypes": ["ANY_PLATFORM"],
            "threatEntryTypes": ["URL"],
            "threatEntries": [{"url": url}]
        }
    }
    try:
        r = requests.post(
            f"https://safebrowsing.googleapis.com/v4/threatMatches:find?key=
{API_KEY}",
            json=body
        )
        return "matches" in r.json()
    except:
        return False
def check_ssl(domain):
    try:

```

```

context = ssl.create_default_context()

with socket.create_connection((domain, 443), timeout=5) as sock:
    with context.wrap_socket(sock, server_hostname=domain) as ssock:
        cert = ssock.getpeercert()
        return True
except:
    return False

def get_ssl_info(hostname):
    context = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
            cert_bin = ssock.getpeercert(binary_form=True)
            cert = ssock.getpeercert()
            import hashlib
            sha256 = hashlib.sha256(cert_bin).hexdigest()
    return {
        "issuer": dict(x[0] for x in cert['issuer']),
        "issued_on": cert['notBefore'],
        "expires_on": cert['notAfter'],
        "fingerprint": sha256
    }

def format_date(date_str):
    return datetime.strptime(date_str, "%b %d %H:%M:%S %Y
%Z").strftime("%b %d, %Y")

def check_ssl_risk(cert):
    expiry_date = datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S
%Y %Z")
    if expiry_date < datetime.now():
        return " ✖ Certificate Expired"

```



```

    return "✅ SSL connection is secure"

def get_whois_info(domain):
    try:
        w = whois.whois(domain)
        return {
            "domain": domain,
            "registrar": w.registrar,
            "creation_date": str(w.creation_date),
            "expiry_date": str(w.expiration_date)
        }
    except:
        return {
            "domain": domain,
            "registrar": "Unknown",
            "creation_date": "Unknown",
            "expiry_date": "Unknown"
        }

def get_ip_address(url):
    try:
        parsed = urlparse(url)
        domain = parsed.netloc.split(":")[0]
        ip = socket.gethostbyname(domain)
        return ip
    except:
        return None

def get_ip_info(ip):
    try:
        res = requests.get(f"http://ip-api.com/json/{ip}", timeout=5)
        data = res.json()

```

```

        return {
            "country": data.get("country"),
            "city": data.get("city"),
            "isp": data.get("isp")
        }
    except:
        return {
            "country": "Unknown",
            "city": "Unknown",
            "isp": "Unknown"
        }

def extract_subdomain_info(url):
    parsed = urlparse(url)
    hostname = parsed.netloc.lower()
    parts = hostname.split(".")
    if len(parts) > 2:
        subdomain = ".".join(parts[:-2])
        main_domain = ".".join(parts[-2:])
    else:
        subdomain = ""
        main_domain = hostname
    return subdomain, main_domain

def detect_homograph(domain):
    suspicious_patterns = [
        ("0", "o"),
        ("1", "l"),
        ("@", "a"),
        ("$", "s"),
        ("rn", "m")
    ]
    for fake, real in suspicious_patterns:

```

```

        if fake in domain:
            return True
    return False

def redirect_analysis(url):
    redirect_count = 0
    domain_changed = False
    try:
        response = requests.get(url, timeout=5, allow_redirects=True)
        redirect_count = len(response.history)
        original_domain = urlparse(url).netloc
        final_domain = urlparse(response.url).netloc
        if original_domain != final_domain:
            domain_changed = True
    except:
        pass
    return redirect_count, domain_changed

def analyze_url(url):
    score = 0
    parsed = urlparse(url)
    domain = parsed.netloc.lower()
    # SSL Check
    if not check_ssl(domain):
        score += 30
    # HTTPS check
    if not url.startswith("https://"):
        score += 20
    # Long URL
    if len(url) > 75:
        score += 15
    # IP address in domain
    if re.search(r'\d+\.\d+\.\d+\.\d+', domain):

```

```

    score += 25
# Suspicious keywords
suspicious_words = [
    "login", "verify", "update", "secure",
    "account", "banking", "confirm",
    "password", "otp", "signin"
]
for word in suspicious_words:
    if word in url.lower():
        score += 10
# Too many dots
if domain.count(".") > 3:
    score += 15
# Trusted domains override
for trusted in TRUSTED_DOMAINS:
    if trusted in domain:
        score = 0
        break
# Phishing domains check
for phish in PHISH_DOMAINS:
    if phish in domain:
        score = 100
        break # immediately return
# Extract subdomain
subdomain, main_domain = extract_subdomain_info(url)
# Suspicious: too many subdomains
if subdomain.count(".") >= 2:
    score += 20
# Suspicious keywords in subdomain
for word in ["login", "secure", "bank", "verify"]:
    if word in subdomain:

```

```

        score += 15
    # Limit score
    if score > 100:
        score = 100
    # Result classification
    if score < 30:
        result = "Safe Website"
    elif score < 70:
        result = "Suspicious Website"
    else:
        result = "Phishing Website"
    return result, score
@app.route("/")
def home():
    return render_template("index.html")
@app.route("/scan", methods=["POST"])
def scan():
    user_url = request.form["url"]
    user_url = normalize_url(user_url)
    result, threat_score = analyze_url(user_url)
    redirect_count, domain_changed = redirect_analysis(user_url)
    subdomain, main_domain = extract_subdomain_info(user_url)
    whois_data = get_whois_info(main_domain)
    vt_result = check_virustotal(user_url)
    google_flag = check_google_safe(user_url)
    ip_address = get_ip_address(user_url)
    if ip_address:
        ip_info = get_ip_info(ip_address)
    else:
        ip_info = {
            "country": "Unknown",

```

```

        "city": "Unknown",
        "isp": "Unknown"
    }
    if vt_result:
        if vt_result["malicious"] > 0:
            threat_score += 40
        elif vt_result["suspicious"] > 0:
            threat_score += 20
    if google_flag:
        threat_score += 50
    if threat_score > 100:
        threat_score = 100

    if threat_score >= 70:
        result = "Phishing Website"
    elif threat_score >= 30:
        result = "Suspicious Website"
    else:
        result = "Safe Website"
    parsed = urlparse(user_url)
    domain = parsed.netloc
    try:
        ssl_data = get_ssl_info(domain)
        ssl_issuer = ssl_data["issuer"].get("organizationName", "Unknown")
        ssl_issued = format_date(ssl_data["issued_on"])
        ssl_expiry = format_date(ssl_data["expires_on"])
        ssl_fingerprint = ssl_data["fingerprint"]
        expiry_date = datetime.strptime(ssl_data["expires_on"], "%b %d
%H:%M:%S %Y %Z")
        if expiry_date < datetime.now():
            ssl_status = "✖ Certificate Expired"

```

```

else:
    ssl_status = "✅ SSL connection is secure"
except:
    ssl_issuer = "Unknown"
    ssl_issued = "N/A"
    ssl_expiry = "N/A"
    ssl_status = "⚠️ Unable to fetch SSL info"
    ssl_fingerprint = "N/A"
if redirect_count > 3 or domain_changed:
    threat_score += 20
    if threat_score > 100:
        threat_score = 100
    if threat_score >= 70:
        result = "Phishing Website"
    else:
        result = "Suspicious Website"
conn = sqlite3.connect("scans.db")
c = conn.cursor()
c.execute(
    "INSERT INTO scans (url, result, threat_score) VALUES (?, ?, ?)",
    (user_url, result, threat_score)
)
conn.commit()
conn.close()
return render_template(
    "result.html",
    url=user_url,
    result=result,
    threat_score=threat_score,
    redirect_count=redirect_count,
    domain_changed=domain_changed,

```

```

        subdomain=subdomain,
        main_domain=main_domain,
        whois=whois_data,
        ip_address=ip_address,
        ip_info=ip_info,
        domain=domain,
        ssl_issuer=ssl_issuer,
        ssl_issued=ssl_issued,
        ssl_expiry=ssl_expiry,
        ssl_status=ssl_status,
        ssl_fingerprint=ssl_fingerprint
    )
@app.route("/admin")
def admin():
    conn = sqlite3.connect("scans.db")
    c = conn.cursor()
    c.execute("SELECT * FROM scans ORDER BY id DESC")
    scans = c.fetchall()
    conn.close()
    return render_template("admin.html", scans=scans)
if __name__ == "__main__":
    app.run(debug=True)

```


CHAPTER 9

SYSTEM TESTING

System testing for the AI-Powered URL Security and Phishing Detection System is essential to ensure that all modules function accurately and efficiently under real-world conditions. This phase validates that the system correctly analyzes user-provided URLs, processes them through feature extraction and internal analysis, and integrates external threat intelligence services such as VirusTotal and Google Safe Browsing to detect malicious and phishing websites.

The testing focuses on verifying proper URL normalization, accurate threat scoring, and correct classification of websites as safe, suspicious, or phishing. It also ensures smooth interaction between different modules such as IP analysis, WHOIS lookup, SSL certificate validation, and redirect analysis. Additionally, the system checks the integration between the Flask-based backend and the web interface to ensure seamless data flow and real-time response.

Overall, system testing confirms that the system performs its intended purpose—detecting phishing and malicious URLs and providing accurate security analysis—reliably, efficiently, and consistently.

9.1 Unit Testing

In the unit testing phase, each component of the AI-Powered URL Security and Phishing Detection System was tested independently. The URL input and normalization modules were verified to handle and standardize URLs correctly. Feature extraction modules, including HTTPS, SSL, domain, and subdomain checks, were tested for accurate detection. The WHOIS module was validated for correct domain information retrieval, and the AI phishing detection model was tested with safe and malicious URLs to ensure reliable threat prediction. The reporting module was checked for proper display of results. All units performed successfully, providing a stable foundation for integration.

9.2 Functional Testing

Functional testing verified that all features of the system work as intended according to the specifications. The URL submission module was tested to ensure it correctly accepts input and triggers backend processes. SSL and HTTPS checks were validated to accurately detect certificate validity and secure protocols. The WHOIS module was tested for precise extraction of domain registration details. Feature extraction and analysis modules were checked to confirm they process data correctly and produce reliable outputs. Each function performed as expected, confirming the system behaves reliably in normal scenarios.

9.3 Integration Testing

Integration testing focused on how modules interact with each other when combined. The URL normalization module was integrated with feature extraction, and data flow between them was monitored to ensure consistency. The HTTPS, SSL, and domain analysis modules were combined to confirm accurate reporting in sequence. Outputs from backend analysis were verified on the user interface to ensure they display correctly. Integration testing confirmed that the system functions smoothly as a cohesive unit rather than separate modules.

9.4 Performance Testing

Performance testing assessed the system's efficiency and stability under various conditions. URL scanning speed was measured to ensure timely results. The system was subjected to multiple simultaneous URL requests to check its ability to handle high loads without crashes. CPU and memory usage were monitored to detect potential bottlenecks. The system maintained consistent response times and resource efficiency, proving it is capable of real-world operation.

9.5 Security Testing

Security testing ensured the system is protected against potential vulnerabilities. Input validation was verified to prevent malicious URL injections. SSL and HTTPS checks correctly identified insecure connections. Backend security, including access control and data handling, was assessed to prevent unauthorized access to sensitive information. Error handling mechanisms were reviewed to ensure no sensitive details are exposed. These checks confirmed that the system maintains robust security standards.

9.6 User Interface Testing

User interface testing confirmed that the interface is intuitive, responsive, and user-friendly. All interactive elements such as buttons, input fields, and navigation links were tested for proper operation across different browsers and screen sizes. Dark and light modes were validated for visual clarity and consistency. The UI successfully guides users through URL scanning and displays results clearly, ensuring a smooth user experience.

9.7 Validation Testing

Validation testing ensured that the system meets end-user requirements and intended objectives. Various safe and malicious URLs were tested to confirm accurate detection and reporting. User feedback was incorporated to improve clarity of results and suggestions. Outputs were cross-checked with trusted external sources to confirm reliability. Validation testing verified that the system fulfills its intended purpose of providing dependable URL security analysis.

CHAPTER 10

CONCLUSION

The AI-Powered URL Security and Phishing Detection System successfully integrates multiple modules to provide real-time analysis of URLs for potential threats. From URL input and normalization to feature extraction, SSL verification, and WHOIS checks, each component functions as intended, forming a reliable pipeline for phishing detection. The AI-based model demonstrated accurate prediction capabilities, effectively distinguishing between safe and malicious URLs.

Integration and system testing confirmed that the modules work seamlessly together, ensuring consistent and accurate results for end users. Performance and security evaluations showed that the system is responsive, stable under load, and robust against common security vulnerabilities. The user interface provides a clear, intuitive experience, making it easy for users to scan URLs and interpret results quickly.

Overall, the system meets its design objectives by combining AI intelligence with practical security checks, offering a dependable tool for phishing prevention. Its modular architecture allows for future enhancements, such as adding new threat detection features or expanding the AI model's dataset, ensuring long-term adaptability and effectiveness in combating evolving online threats.

CHAPTER 11

FUTURE ENHANCEMENT

One potential enhancement for the AI-Powered URL Security and Phishing Detection System is to extend its capabilities beyond URL scanning to include file uploads for phishing detection. Users could upload documents such as PDFs, Word files, or spreadsheets, and the system could analyze embedded links, macros, or suspicious content to detect potential phishing attempts. This would broaden the system's scope, making it useful for organizations and individuals who frequently receive files containing hidden threats.

Another improvement could be to develop a browser extension version of the system. This would allow real-time URL scanning as users browse the web, providing instant alerts for malicious websites without the need to manually enter URLs into the web application. Integrating the AI detection engine into popular browsers would make the system more accessible and convenient, increasing user adoption and enhancing online safety.

CHAPTER 12

REFERENCES

1. Ume Zara, Kashif Ayyub, Hikmat Ullah Khan, Ali Daud, Tariq Alsahfi, Saima Gulzar Ahmad, "Phishing Website Detection Using Deep Learning Models," IEEE Access, vol. 12, no. 2, Oct. 2024, DOI: 10.1109/ACCESS.2024.3486462.
2. K. Jain and B. B. Gupta, "PHISH-SAFE: URL Features-Based Phishing Detection System Using Machine Learning," in Cyber Security (Advances in Intelligent Systems and Computing), vol. 729, Singapore: Springer, 2018, pp. 467–474, DOI: 10.1007/978-981-10-8536-9_44.
3. J. K. Lee, Y. Chang, H. Y. Kwon and B. Kim, "Reconciliation of Privacy with Preventive Cybersecurity: The Bright Internet Approach," Information Systems Frontiers, vol. 22, no. 1, pp. 45–57, Feb. 2020, DOI: 10.1007/s10796-020-09984-5.
4. S. Asiri, Y. Xiao, S. Alzahrani, S. Li and T. Li, "A Survey of Intelligent Detection Designs of HTML URL Phishing Attacks," IEEE Access, vol. 11, pp. 6421–6443, 2023, DOI: 10.1109/ACCESS.2023.3237798.
5. N. A. Azeez, S. Misra, I. A. Margaret, L. Fernandez-Sanz and S. M. Abdulhamid, "Adopting Automated Whitelist Approach for Detecting Phishing Attacks," Computers & Security, vol. 108, Sep. 2021, Art. no. 102328, DOI: 10.1016/j.cose.2021.102328.
6. J. Ma, L. K. Saul, S. Savage and G. M. Voelker, "Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs," in Proc. ACM SIGKDD, 2009, pp. 1245–1254.
7. N. Abdelhamid, A. Ayeshe and F. Thabtah, "Phishing Detection Based Associative Classification Data Mining," Expert Systems with Applications, vol. 41, no. 13, pp. 5948–5959, 2014.
8. R. Basnet, A. H. Sung and Q. Liu, "Learning to Detect Phishing URLs," International Journal of Research in Engineering and Technology, vol. 3, no. 6, pp. 11–24, 2014.

9. O. K. Sahingoz, E. Buber, O. Demir and B. Diri, "Machine Learning Based Phishing Detection from URLs," *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
10. R. S. Rao and A. R. Pais, "Detection of Phishing Websites Using an Efficient Feature-Based Machine Learning Framework," *Neural Computing and Applications*, vol. 31, no. 8, pp. 3851–3873, 2019.
11. R. Verma and A. Das, "What's in a URL: Fast Feature Extraction and Malicious URL Detection," in *Proc. IEEE Int. Conf.*, 2017, pp. 1–6.
12. A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas and F. A. Gonzalez, "Classifying Phishing URLs Using Recurrent Neural Networks," in *IEEE Conf. Intelligence and Security Informatics*, 2017, pp. 1–6.
13. S. Marchal, K. Saari, N. Singh and N. Asokan, "Know Your Phish: Novel Techniques for Detecting Phishing Sites and Their Targets," in *IEEE Int. Conf. Distributed Computing Systems*, 2016, pp. 323–333.
14. R. M. Mohammad, F. Thabtah and L. McCluskey, "Predicting Phishing Websites Based on Self-Structuring Neural Network," *Neural Computing and Applications*, vol. 25, no. 2, pp. 443–458, 2014.
15. Y. Zhang, J. I. Hong and L. F. Cranor, "Cantina: A Content-Based Approach to Detecting Phishing Web Sites," in *Proc. WWW Conf.*, 2007, pp. 639–648.



A DIGITAL LIBRARY

WWW.IJISRT.COM

INTERNATIONAL JOURNAL OF INNOVATIVE SCIENCE AND RESEARCH TECHNOLOGY

IJISRT A DIGITAL LIBRARY

AUTHOR CERTIFICATE

THIS IS TO CERTIFY THAT THE MANUSCRIPT, TITLED
AI-Powered URL Security and Phishing Detection System

AUTHORED BY
Abishek G. P.

HAS BEEN PUBLISHED IN
Volume 11 | Issue 3 | March-2026

ARTICLE DIGITAL NO.
IJISRT26MAR1467



EDITOR IN CHIEF IJISRT

This document certifies that the manuscript listed above was submitted by above said respected author
To verify the submitted manuscript please visit our official website: www.ijisrt.com
Or Email us: editor@ijisrt.com



A DIGITAL LIBRARY

WWW.IJISRT.COM

INTERNATIONAL JOURNAL OF INNOVATIVE SCIENCE AND RESEARCH TECHNOLOGY

IJISRT A DIGITAL LIBRARY

AUTHOR CERTIFICATE

THIS IS TO CERTIFY THAT THE MANUSCRIPT, TITLED
AI-Powered URL Security and Phishing Detection System

AUTHORED BY
Akash A.

HAS BEEN PUBLISHED IN
Volume 11 | Issue 3 | March-2026

ARTICLE DIGITAL NO.
IJISRT26MAR1467



EDITOR IN CHIEF IJISRT

This document certifies that the manuscript listed above was submitted by above said respected author
To verify the submitted manuscript please visit our official website: www.ijisrt.com
Or Email us: editor@ijisrt.com



A DIGITAL LIBRARY

WWW.IJISRT.COM

INTERNATIONAL JOURNAL OF INNOVATIVE SCIENCE AND RESEARCH TECHNOLOGY

IJISRT A DIGITAL LIBRARY

AUTHOR CERTIFICATE

THIS IS TO CERTIFY THAT THE MANUSCRIPT, TITLED
AI-Powered URL Security and Phishing Detection System

AUTHORED BY
Eswar M. S.

HAS BEEN PUBLISHED IN
Volume 11 | Issue 3 | March-2026

ARTICLE DIGITAL NO.
IJISRT26MAR1467



EDITOR IN CHIEF IJISRT

This document certifies that the manuscript listed above was submitted by above said respected author
To verify the submitted manuscript please visit our official website: www.ijisrt.com
Or Email us: editor@ijisrt.com